

為網頁應用程式新增推播通知

來源：<https://codelabs.developers.google.com/codelabs/push-notifications?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何在網頁應用程式中加入推播通知。

目錄

1. 總覽
2. 做好準備
3. 註冊 Service Worker
4. 初始化狀態
5. 訂閱使用者
6. 處理權限遭拒的情況
7. 處理推送事件
8. 通知點擊
9. 傳送推播訊息
10. 取消訂閱使用者
11. 已完成

步驟 1 · 約 0 分鐘

1. 總覽

推送訊息是簡單又有效的方法，可讓使用者再次與您互動。在本程式碼研究室中，您將瞭解如何將推播通知新增至網頁應用程式。

課程內容

- 如何訂閱及取消訂閱使用者的推播訊息
- 如何處理收到的推播訊息
- 如何顯示通知
- 如何回應通知點擊

軟硬體需求

- Chrome 52 以上版本
 - [Web Server for Chrome](#)，或您選擇的網路伺服器
 - 文字編輯器
 - 具備 HTML、CSS、JavaScript 和 Chrome 開發人員工具的基本知識
 - 程式碼範例 (請參閱「開始設定」一節)
-

步驟 2 · 約 0 分鐘

2. 做好準備

下載程式碼範例

您可以透過下列兩種方式取得本程式碼研究室的程式碼範例：

- 複製 Git 存放區：

```
git clone https://github.com/GoogleChrome/push-notifications.git
```

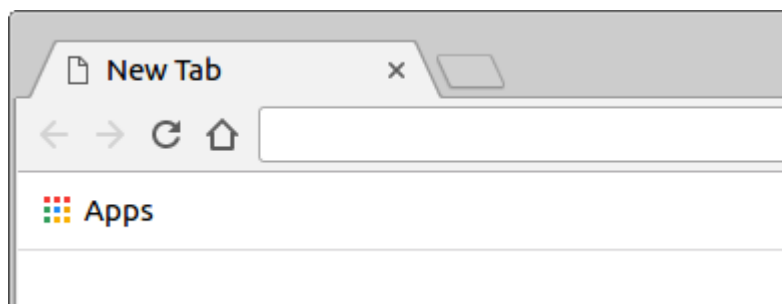
- 下載 ZIP 檔案：

如果將來源下載為 ZIP 檔案，解壓縮後會得到根資料夾 `push-notifications-master`。

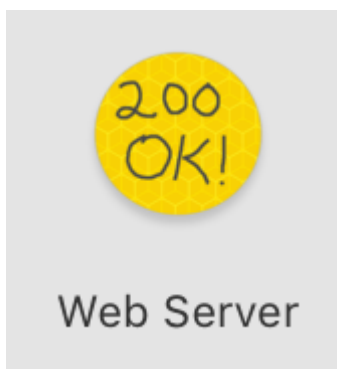
安裝及驗證網路伺服器

您可以自由使用自己的網路伺服器，但本程式碼研究室的設計與 Chrome 專用的網路伺服器應用程式相容。如果尚未安裝該應用程式，請前往 Chrome 線上應用程式商店下載：

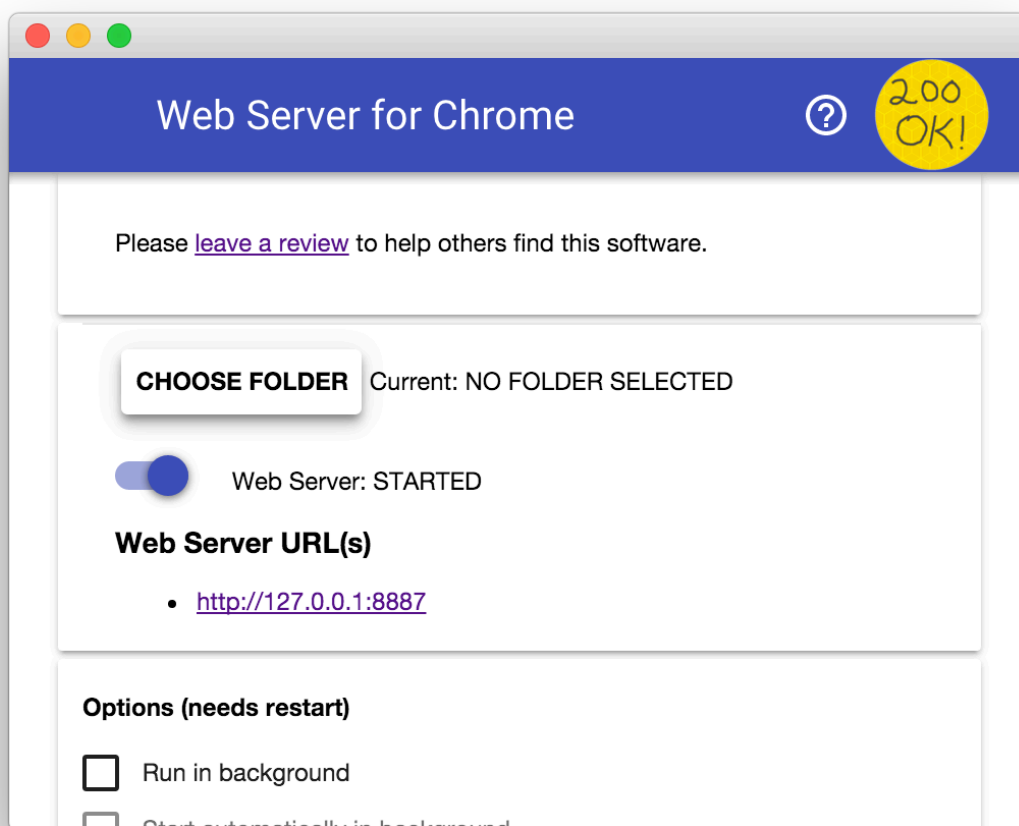
安裝 Web Server for Chrome 應用程式後，請按一下書籤列上的「應用程式」捷徑：



在「應用程式」視窗中，按一下「網頁伺服器」圖示：



接著會看到這個對話方塊，可供您設定本機網路伺服器：



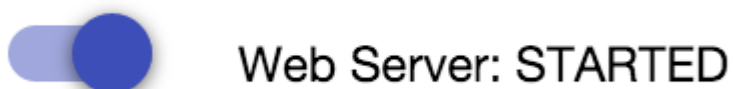
按一下「選擇資料夾」按鈕，然後選取您下載的 `push-notifications` 資料夾中的 `app` 資料夾。這樣一來，您就能透過對話方塊「Web Server URL(s)」部分顯示的網址，提供正在進行的工作。

在「選項」下方，勾選「自動顯示 index.html」旁的方塊，如下所示：

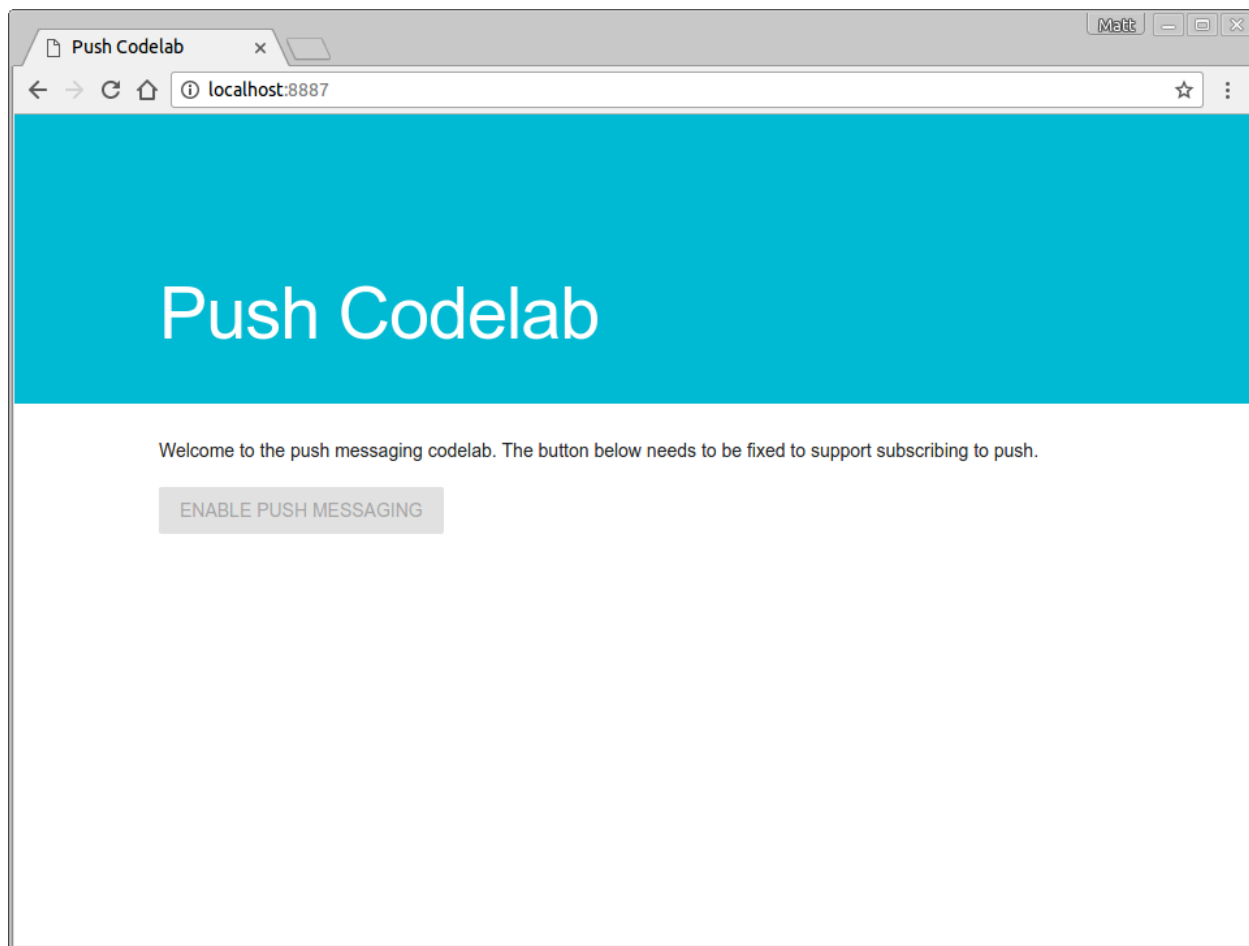
Options (needs restart)

- ☐ Run in background
- ☐ Start on login
- ☐ Accessible on local network
- ☐ Also on internet
- ☐ Prevent computer from sleeping
- ☒ Automatically show index.html

接著將「Web Server: STARTED」切換鈕滑動至左側，然後再滑動至右側，停止並重新啟動伺服器。



按一下「Web Server URL」，在網路瀏覽器中造訪網站。您應該會看到類似下方的頁面，但您的版本可能會顯示 127.0.0.1:8887 做為地址：

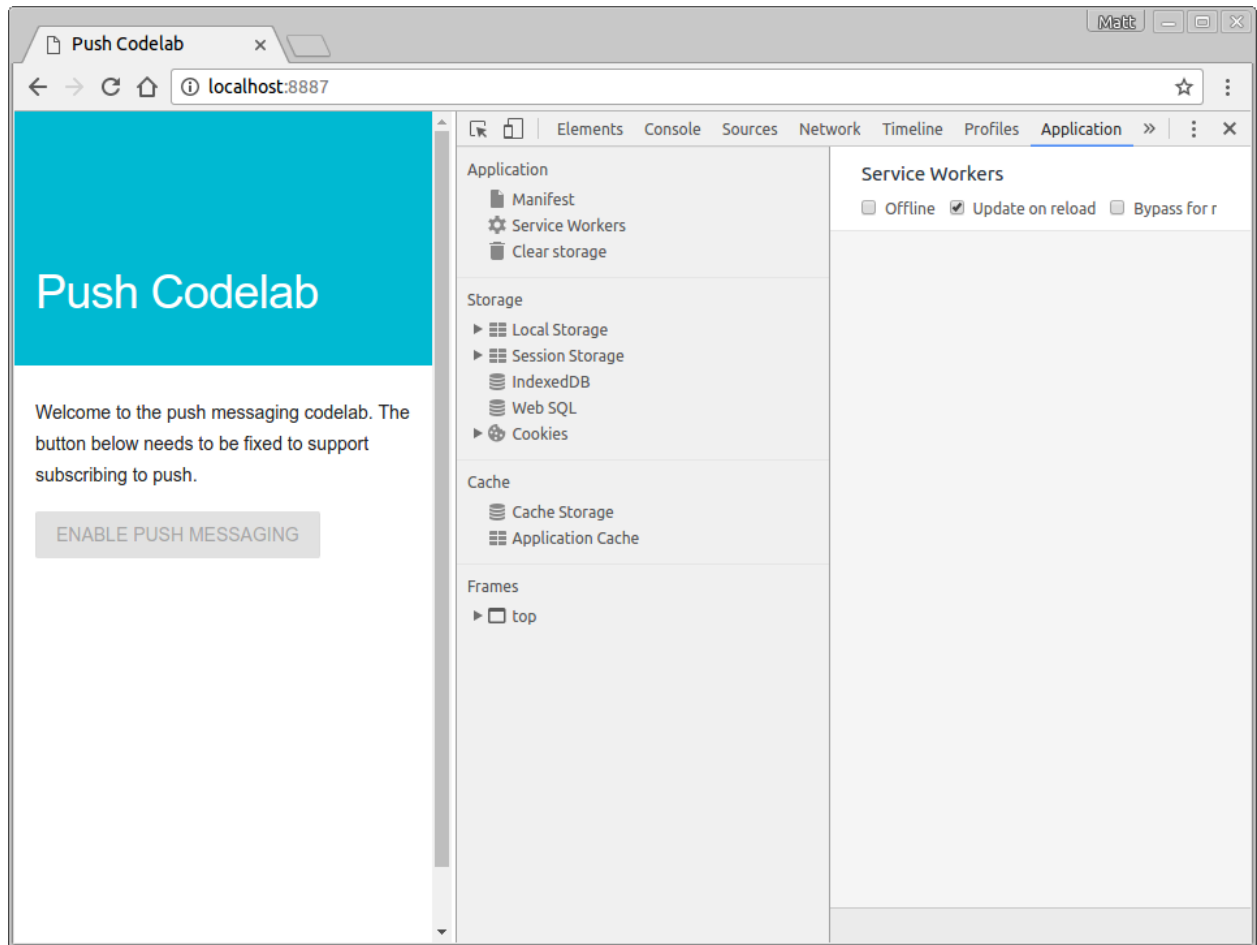


一律更新 Service Worker

在開發期間，請務必確保 Service Worker 隨時保持最新狀態，並採用最新異動。

如要在 Chrome 中設定這項功能，請按照下列步驟操作：

1. 前往「Push Codelab」分頁。
2. 開啟開發人員工具：在 Windows 和 Linux 上按 Ctrl-Shift-I，在 macOS 上按 Cmd-Option-I。
3. 選取「應用程式」面板，按一下「服務工作人員」分頁，然後勾選「重新載入時更新」核取方塊。啟用這個核取方塊後，每次重新載入網頁時，系統都會強制更新 Service Worker。



步驟 3 · 約 0 分鐘

3. 註冊 Service Worker

完成的程式碼

在 `app` 目錄中，您會看到名為 `sw.js` 的空白檔案。這個檔案就是 Service Worker。目前可以留空。稍後您會加入程式碼。

首先，您需要將這個檔案註冊為 Service Worker。

`app/index.html` 載入網頁 `scripts/main.js`。您可以在這個 JavaScript 檔案中註冊 Service Worker。

在 `scripts/main.js` 加入以下程式碼：

```
if ('serviceWorker' in navigator && 'PushManager' in window) {
  console.log('Service Worker and Push are supported');

  navigator.serviceWorker.register('sw.js')
    .then(function(swReg) {
      console.log('Service Worker is registered', swReg);

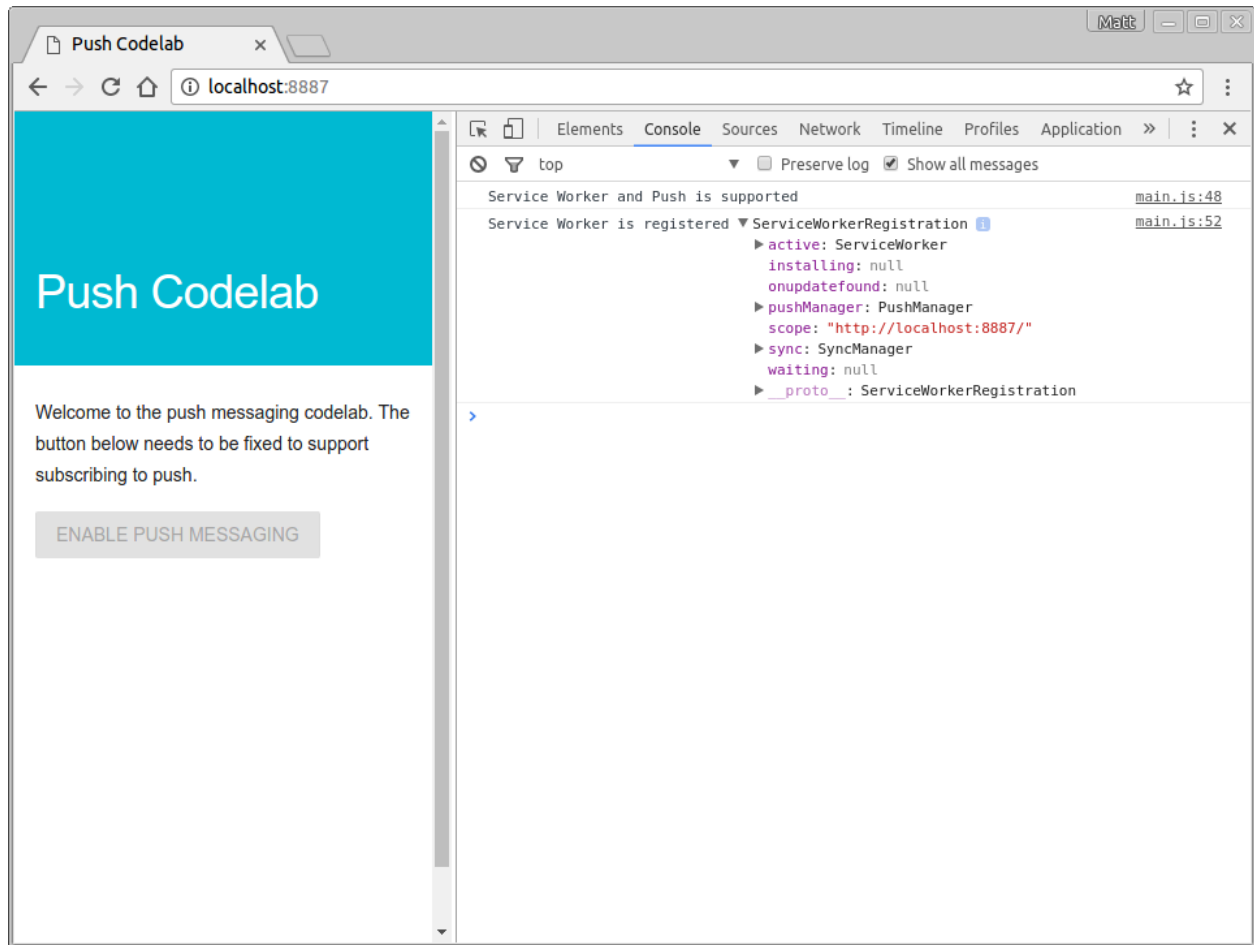
      swRegistration = swReg;
    })
    .catch(function(error) {
      console.error('Service Worker Error', error);
    });
} else {
  console.warn('Push messaging is not supported');
  pushButton.textContent = 'Push Not Supported';
}
```

這段程式碼會檢查瀏覽器是否支援 Service Worker 和推播訊息。如果支援，程式碼會註冊 `sw.js` 檔案。

馬上試試

在瀏覽器中重新整理「Push Codelab」分頁，確認變更是否生效。

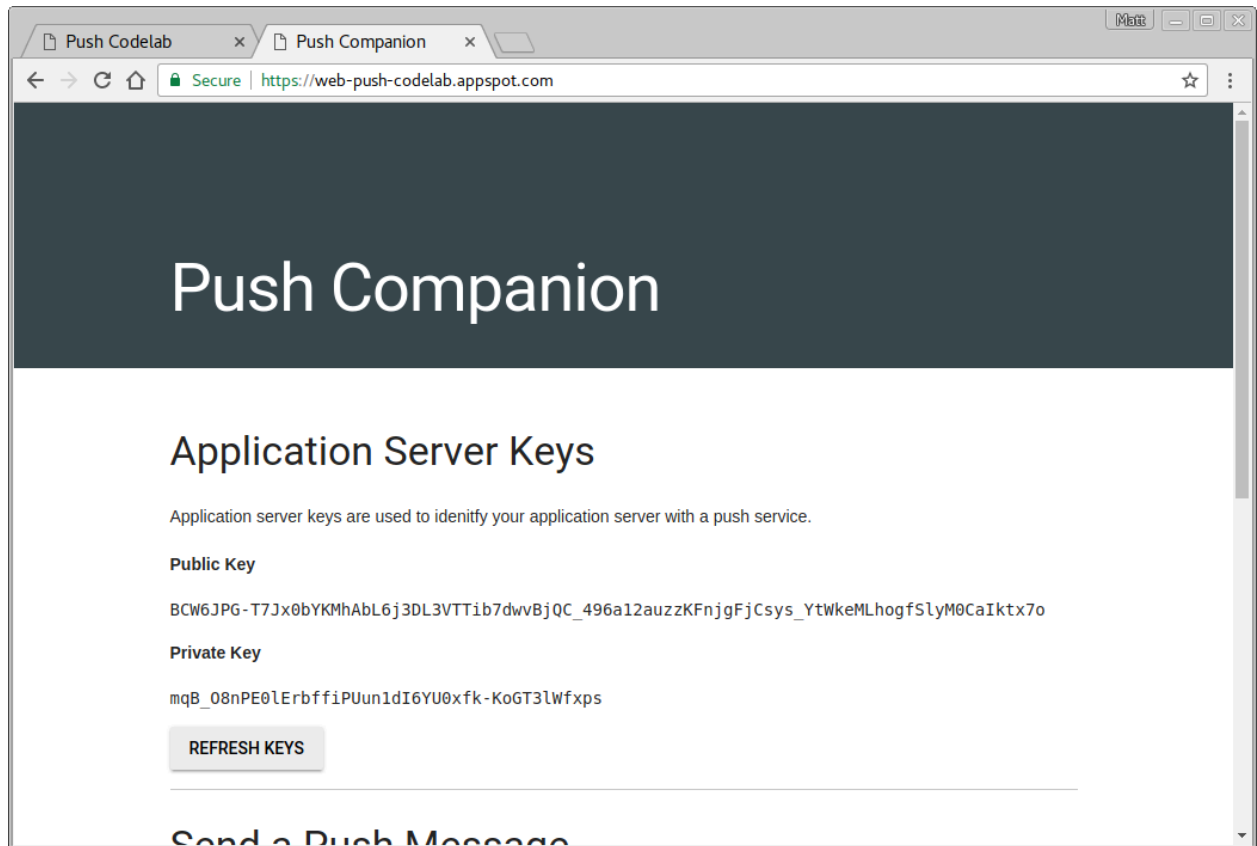
在 Chrome 開發人員工具中檢查主控台是否有 `Service Worker is registered message`，如下所示：



取得應用程式伺服器金鑰

如要使用本程式碼研究室，您需要產生應用程式伺服器金鑰。您可以在隨附網站上執行這項操作：web-push-codelab.glitch.me

您可以在這裡產生公開和私密金鑰組。



將公開金鑰複製到 `scripts/main.js`，取代 `<Your Public Key>` 值：

```
const applicationServerPublicKey = '<Your Public Key>';
```

重要事項：請勿在網頁應用程式中放置私密金鑰！

步驟 4 · 約 0 分鐘

4. 初始化狀態

完成的程式碼

目前網頁應用程式的「啟用」按鈕已停用，無法點選。這是因為預設停用推送按鈕是最佳做法，確認瀏覽器支援推送訊息，且您能夠檢查使用者目前是否已訂閱訊息後，再啟用按鈕。

您需要在 `scripts/main.js` 中建立兩個函式：

- `initializeUI`，檢查使用者目前是否已訂閱
- `updateBtn`，根據使用者是否訂閱來啟用按鈕並變更文字

在 `main.js` 中新增 `initializeUI` 函式，如下所示：

```
function initializeUI() {
  // Set the initial subscription value
  swRegistration.pushManager.getSubscription()
    .then(function(subscription) {
      isSubscribed = !(subscription === null);

      if (isSubscribed) {
        console.log('User IS subscribed.');
```

新方法會使用上一步驟中的 `swRegistration`，從中取得 `pushManager` 屬性，並對該屬性呼叫 `getSubscription()`。

`pushManager` 的 `getSubscription()` 會傳回 `promise`，如果目前有訂閱項目，就會使用該項目解析 `promise`。否則會傳回 `null`。這樣一來，您就能檢查使用者是否已訂閱、設定 `isSubscribed` 的值，然後呼叫 `updateBtn()` 更新按鈕。

將 `updateBtn()` 函式新增至 `main.js`：

```
function updateBtn() {
  if (isSubscribed) {
    pushButton.textContent = 'Disable Push Messaging';
  } else {
    pushButton.textContent = 'Enable Push Messaging';
  }

  pushButton.disabled = false;
}
```

這項函式會啟用按鈕，並根據使用者是否已訂閱來變更按鈕文字。

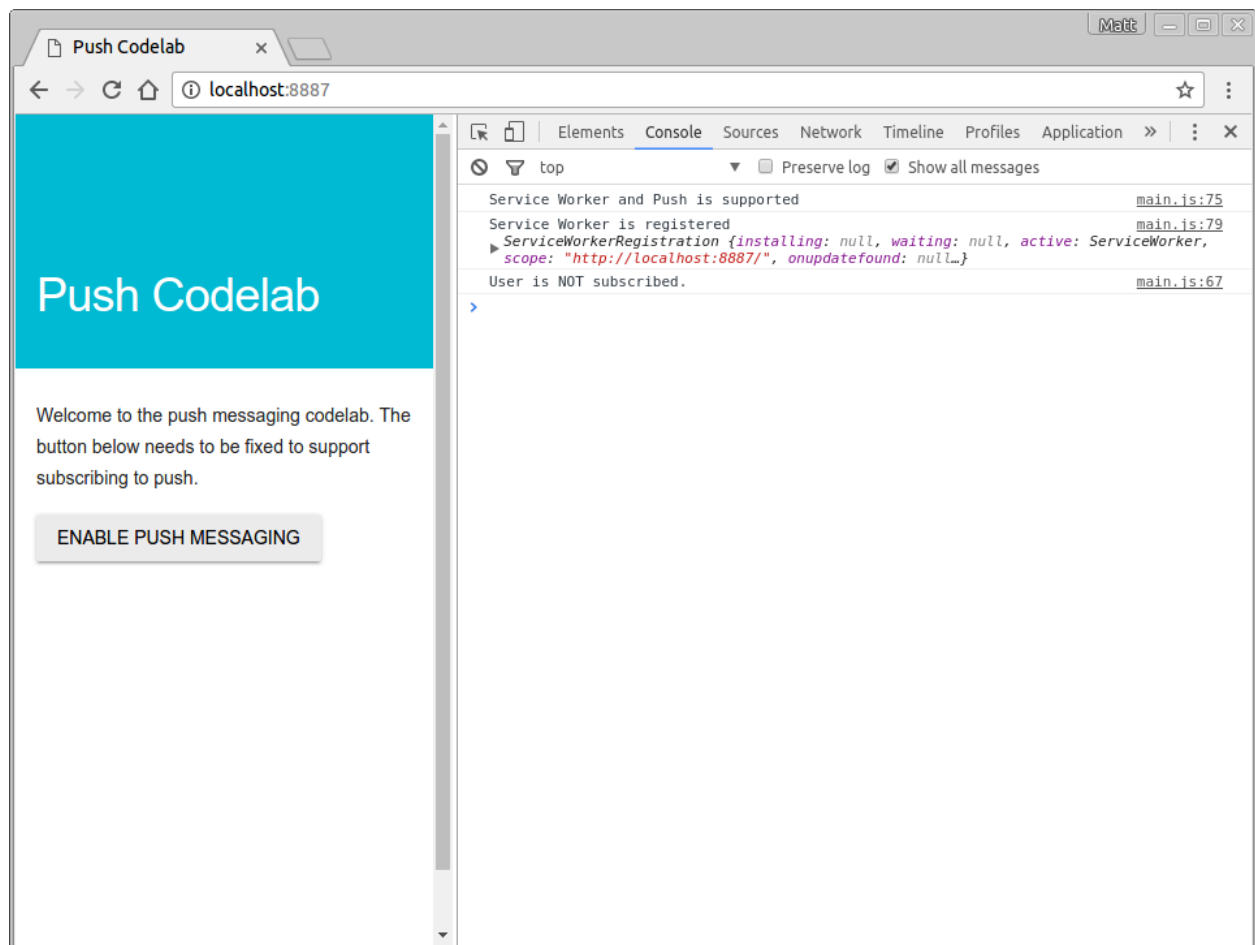
最後一件事是在 Service Worker 註冊 `main.js` 時呼叫 `initializeUI()`：

```
navigator.serviceWorker.register('sw.js')
.then(function(swReg) {
  console.log('Service Worker is registered', swReg);

  swRegistration = swReg;
  initializeUI();
})
```

馬上試試

重新整理「Push Codelab」分頁。您應該會看到「啟用即時訊息」按鈕現在已啟用 (您可以點選)，且控制台中會顯示 `User is NOT subscribed`。



在本程式碼研究室的後續步驟中，每當您訂閱或取消訂閱時，應該會看到按鈕文字變更。

步驟 5 · 約 0 分鐘

5. 訂閱使用者

完成的程式碼

目前「啟用即時訊息」按鈕的功能不多，讓我們一起解決這個問題！

在 `initializeUI()` 函式中，為按鈕新增點擊事件監聽器：

```
function initializeUI() {
  pushButton.addEventListener('click', function() {
    pushButton.disabled = true;
    if (isSubscribed) {
      // TODO: Unsubscribe user
    } else {
      subscribeUser();
    }
  });

  // Set the initial subscription value
  swRegistration.pushManager.getSubscription()
    .then(function(subscription) {
      isSubscribed = !(subscription === null);

      updateSubscriptionOnServer(subscription);

      if (isSubscribed) {
        console.log('User IS subscribed.');
```

使用者點選按鈕時，請停用按鈕，確保使用者不會再次點選，因為訂閱即時訊息可能需要一些時間。

如果使用者目前未訂閱，則呼叫 `subscribeUser()`。為此，您需要在 `scripts/main.js` 中貼上下列程式碼：

```
function subscribeUser() {
  const applicationServerKey = urlB64ToUint8Array(applicationServerPublicKey);
  swRegistration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: applicationServerKey
  })
  .then(function(subscription) {
    console.log('User is subscribed.');


updateSubscriptionOnServer(subscription);



isSubscribed = true;



updateBtn();
  })
  .catch(function(error) {
    console.error('Failed to subscribe the user: ', error);
    updateBtn();
  });
}


```

讓我們逐步瞭解這段程式碼的用途，以及如何為使用者訂閱推播訊息。

首先，您要取得應用程式伺服器的公開金鑰 ([Base64 網址安全編碼](#))，並轉換為 `Uint8Array`，因為這是 `subscribe()` 呼叫的預期輸入內容。`urlB64ToUint8Array()` 函式位於 `scripts/main.js` 的頂端。

轉換值後，請在服務工作人員的 `pushManager` 上呼叫 `subscribe()` 方法，並傳遞應用程式伺服器的公開金鑰和值 `userVisibleOnly: true`。

```
const applicationServerKey = urlB64ToUint8Array(applicationServerPublicKey);
swRegistration.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: applicationServerKey
})
```

`userVisibleOnly` 參數可確保每次傳送推播訊息時，系統都會顯示通知。目前必須提供這個值，且必須為 `true`。

呼叫 `subscribe()` 會傳回 `Promise`，在完成下列步驟後會解析：

1. 使用者已授予顯示通知的權限。
2. 瀏覽器已向推送服務傳送網路要求，以取得產生 `PushSubscription` 所需的資料。

如果這些步驟成功，`subscribe()` `Promise` 會以 `PushSubscription` 解決。如果使用者未授予權限，或訂閱使用者時發生任何問題，`Promise` 會拒絕並傳回錯誤。這會在程式碼研究室中提供下列 `Promise` 鏈結：

```

swRegistration.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: applicationServerKey
})
.then(function(subscription) {
  console.log('User is subscribed.');
```



```

  updateSubscriptionOnServer(subscription);

  isSubscribed = true;

  updateBtn();

})
.catch(function(err) {
  console.log('Failed to subscribe the user: ', err);
  updateBtn();
});

```

這樣一來，您就能取得訂閱項目，並將使用者視為已訂閱，或是接收錯誤並記錄到控制台。在這兩種情況下，您都會呼叫 `updateBtn()`，確保按鈕重新啟用並顯示適當的文字。

在實際應用程式中，您會透過 `updateSubscriptionOnServer()` 函式將訂閱資料傳送至後端，但在本程式碼研究室中，您只需在 UI 中顯示訂閱項目即可。在 `scripts/main.js` 中新增下列函式：

```

function updateSubscriptionOnServer(subscription) {
  // TODO: Send subscription to application server

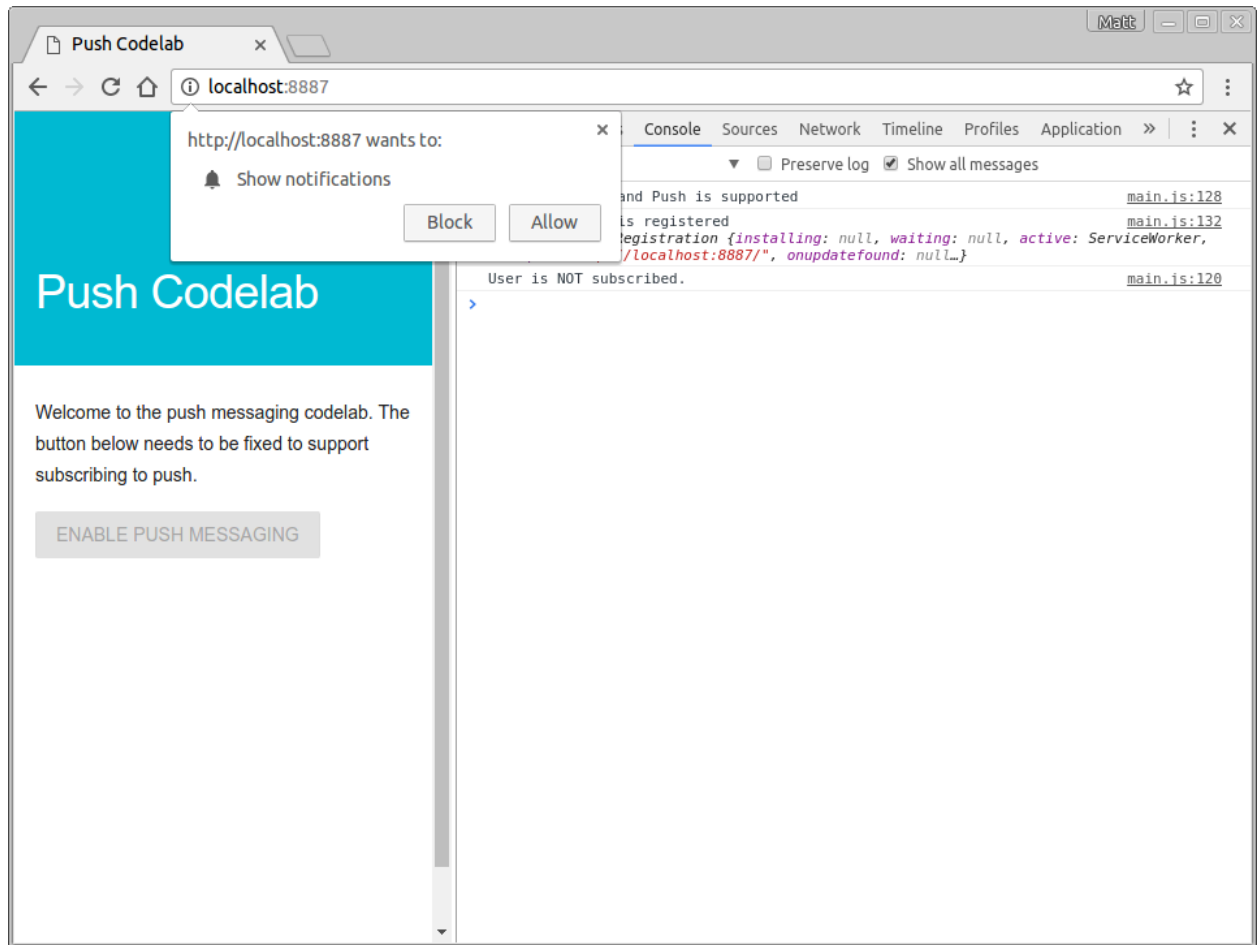
  const subscriptionJson = document.querySelector('.js-subscription-json');
  const subscriptionDetails =
    document.querySelector('.js-subscription-details');

  if (subscription) {
    subscriptionJson.textContent = JSON.stringify(subscription);
    subscriptionDetails.classList.remove('is-invisible');
  } else {
    subscriptionDetails.classList.add('is-invisible');
  }
}

```

馬上試試

前往「Push Codelab」分頁，重新整理頁面，然後按一下按鈕。這時畫面上會出現類似下圖的權限提示：



如果授予權限，控制台應該會記錄 `User is subscribed`。按鈕文字會變更為「停用推播訊息」，且您可以在頁面底部以 JSON 資料格式查看訂閱項目。

Push Codelab

localhost:8887

DISABLE PUSH MESSAGING

Once you've subscribed your user, you'd send their subscription to your server to store in a database so that when you want to send a message you can lookup the subscription and send a message to it.

To simplify things for this code lab copy the following details into the [Push Companion Site](#) and it'll send a push message for you, using the application server keys on the site - so make sure they match.

```
{ "endpoint": "https://fcm.googleapis.com/fcm/send/dSgsCG3zN0w:APA91bGKha74M_3WyHzahlvcybFulevHNCAwEo2xtSfxSpK0KjegA90eaNe7dXai2kl234JCrzuX0WQBqdwTQk_040YdhbAy3rfDIsvJG81pR7CKmEpDsF4jUq_XUM3VdNK9o8Kzq792", "keys": { "p256dh": "BIOCS07Lkc4LDR6qLTAoSq5hrANrKeeFYLPHayZBS4Y56R3ffW_MkJzAAhjtV4-Zc9p2b1aUNK9NY7wPuxcQMq8=", "auth": "4PmS_yD0v402rLGogc0RCA==" } }
```

Elements Console Sources Network Timeline Profiles Application

top

Service Worker and Push is supported

Service Worker is registered

ServiceWorkerRegistration {installing: null, waiting: null, active: ServiceWorker, scope: "http://localhost:8887/", onupdatefound: null...}

User is NOT subscribed.

User is subscribed.

6. 處理權限遭拒的情況

完成的程式碼

您尚未處理的一件事是，如果使用者封鎖權限要求，會發生什麼情況。這需要特別注意，因為如果使用者封鎖權限，網頁應用程式就無法再次顯示權限提示，也無法訂閱使用者。您至少需要停用按鈕，讓使用者知道無法使用。

處理這個情境的明顯位置是 `updateBtn()` 函式。您只需要檢查 `Notification.permission` 值，如下所示：

```
function updateBtn() {
  if (Notification.permission === 'denied') {
    pushButton.textContent = 'Push Messaging Blocked';
    pushButton.disabled = true;
    updateSubscriptionOnServer(null);
    return;
  }

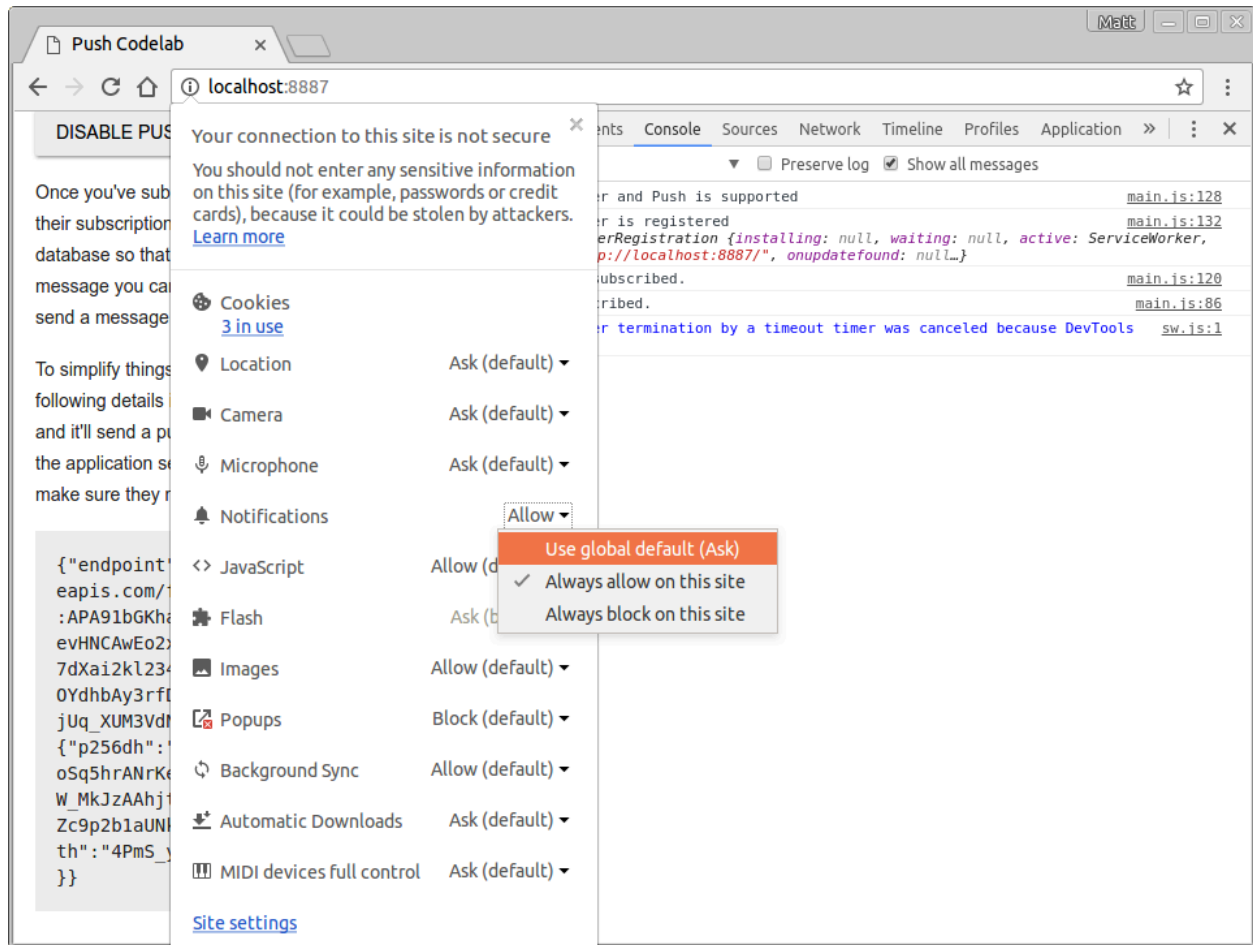
  if (isSubscribed) {
    pushButton.textContent = 'Disable Push Messaging';
  } else {
    pushButton.textContent = 'Enable Push Messaging';
  }

  pushButton.disabled = false;
}
```

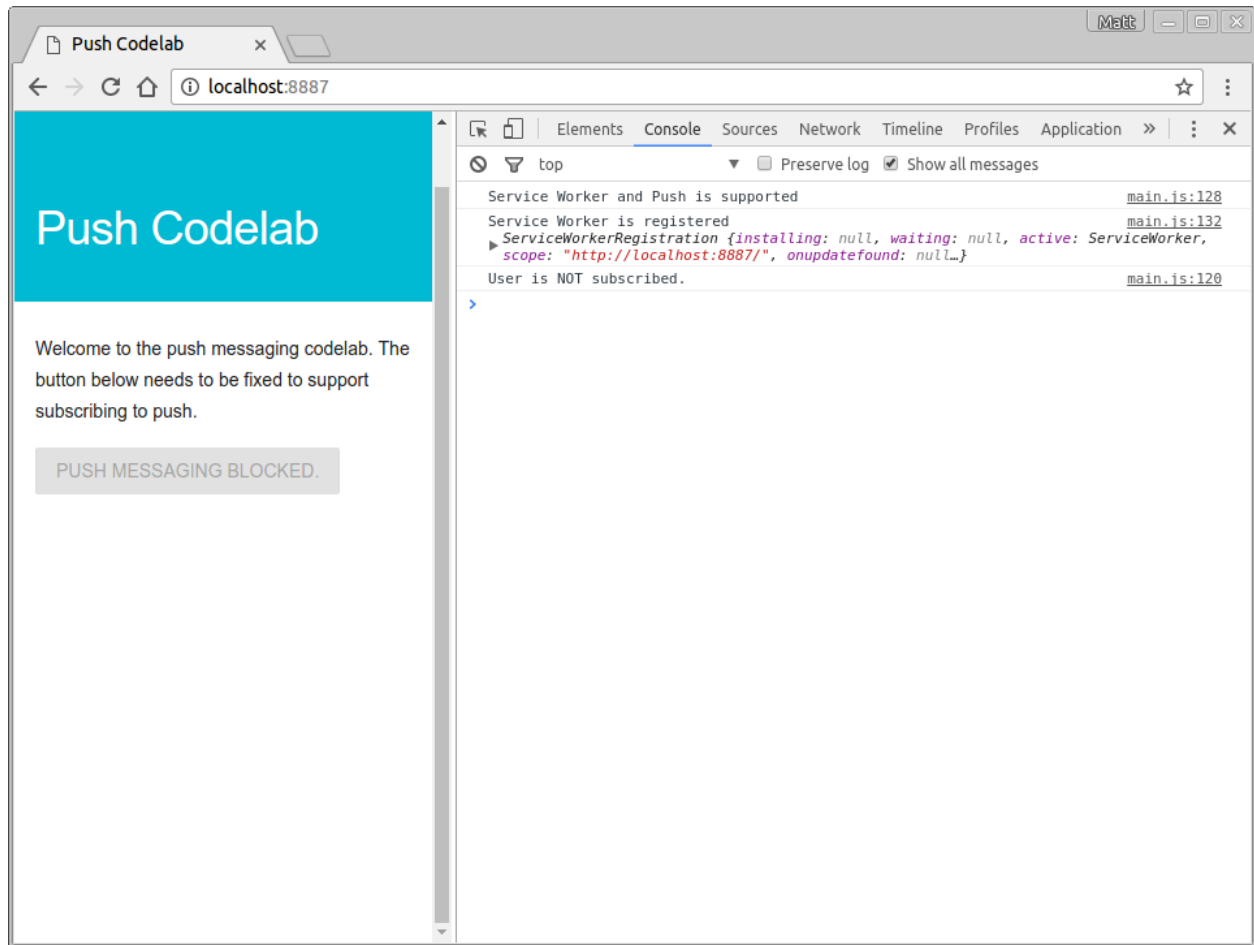
您知道如果權限為 `denied`，使用者就無法訂閱，您也無能為力，因此永久停用按鈕是最佳做法。

馬上試試

由於您已在先前的步驟中授權網頁應用程式，因此請點選網址列中圓圈內的 **i**，然後將「通知」權限變更為「使用全域預設值 (詢問)」。



變更這項設定後，請重新整理頁面，然後按一下「啟用推播訊息」按鈕，並在權限對話方塊中選取「封鎖」。按鈕會停用，並顯示「已封鎖即時訊息」文字。



完成這項變更後，您現在可以訂閱使用者，並處理可能的權限情境。

7. 處理推送事件

完成的程式碼

瞭解如何從後端傳送推播訊息前，請先思考訂閱使用者收到推播訊息時，實際會發生什麼情況。

觸發推播訊息時，瀏覽器會收到推播訊息，找出推播訊息適用的 Service Worker，喚醒該 Service Worker，然後分派推播事件。您需要監聽這項事件，並顯示通知。

在 `sw.js` 檔案中新增下列程式碼：

```
self.addEventListener('push', function(event) {
  console.log('[Service Worker] Push Received.');
```

```
  console.log(`[Service Worker] Push had this data: "${event.data.text()}"`);

  const title = 'Push Codelab';
  const options = {
    body: 'Yay it works.',
    icon: 'images/icon.png',
    badge: 'images/badge.png'
  };

  event.waitUntil(self.registration.showNotification(title, options));
});
```

讓我們逐步瞭解這段程式碼。在 Service Worker 中新增事件監聽器，即可監聽 `push` 事件：

```
self.addEventListener('push', ... );
```

(除非您之前使用過 Web Worker，否則 `self` 可能是新功能。在 Service Worker 檔案中，`self` 會參照 Service Worker 本身)。

收到推播訊息時，系統會呼叫事件監聽器，而您則可呼叫 Service Worker 的 `registration` 屬性上的 `showNotification()`，建立通知。`showNotification()` 需要 `title`，您也可以提供 `options` 物件來設定內文訊息、圖示和徽章。(撰寫本文時，徽章僅適用於 Android)。

```
const title = 'Push Codelab';
const options = {
  body: 'Yay it works.',
  icon: 'images/icon.png',
  badge: 'images/badge.png'
};
self.registration.showNotification(title, options);
```

`push` 事件處理的最後一個主題是 `event.waitUntil()`。這個方法會採用 Promise，讓瀏覽器保持 Service Worker 的運作，直到傳入的 Promise 已解決為止。

為了方便理解上述程式碼，您可以改寫如下：

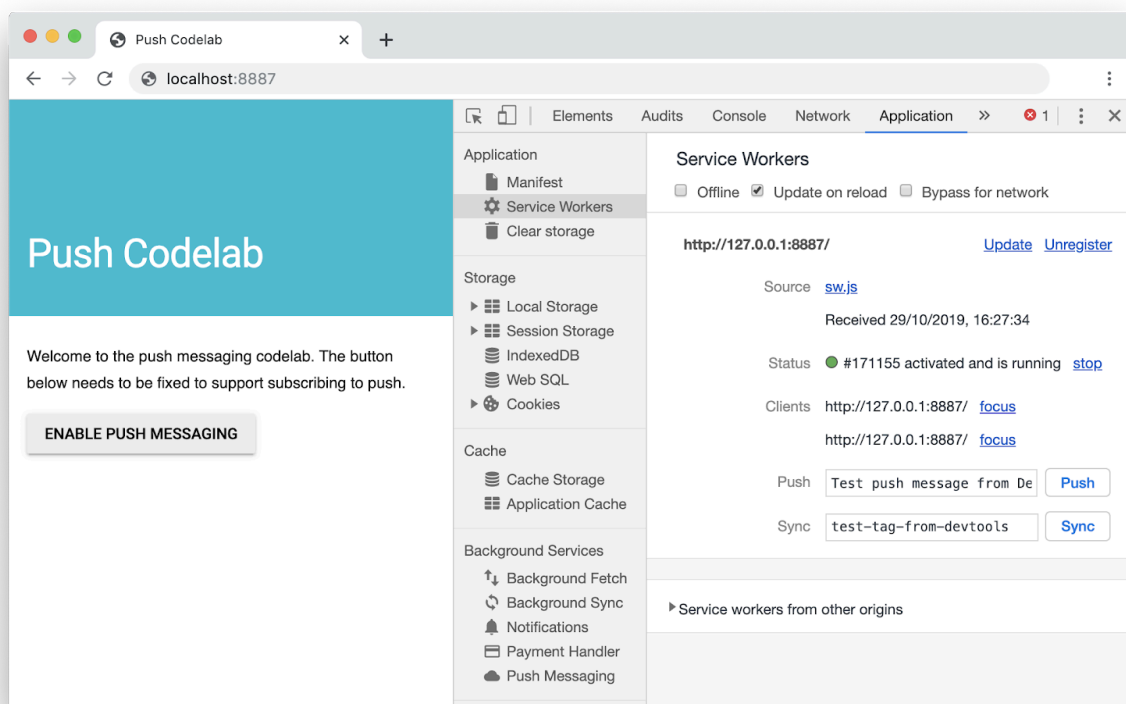
```
const notificationPromise = self.registration.showNotification(title, options);
event.waitUntil(notificationPromise);
```

現在您已逐步瞭解推送事件，接下來要測試推送事件。

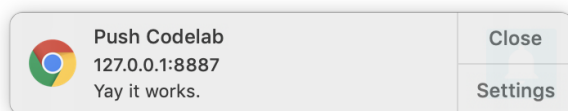
馬上試試

透過 Service Worker 中的推送事件處理功能，您可以觸發假的推送事件，測試收到訊息時會發生什麼情況。

在網頁應用程式中訂閱推播訊息，並確認控制台中顯示 `User IS subscribed`。在開發人員工具的「應用程式」面板中，按一下「服務工作人員」分頁下方的「推送」按鈕：



按一下「推送」後，您應該會看到類似以下的通知：



注意：如果這個步驟無效，請嘗試使用開發人員工具「應用程式」面板中的「取消註冊」連結取消註冊 Service Worker，等待 Service Worker 停止運作，然後重新載入頁面。

步驟 8 · 約 0 分鐘

8. 通知點擊

完成的程式碼

如果您點選其中一則通知，會發現沒有任何反應。您可以在 Service Worker 中監聽 `notificationclick` 事件，處理通知點擊事件。

首先，請在 `sw.js` 中新增 `notificationclick` 監聽器：

```
self.addEventListener('notificationclick', function(event) {  
  console.log('[Service Worker] Notification click received.');  
  event.notification.close();  
  
  event.waitUntil(  
    clients.openWindow('https://developers.google.com/web')  
  );  
});
```

使用者點選通知時，系統會呼叫 `notificationclick` 事件監聽器。

程式碼會先關閉點選的通知：

```
event.notification.close();
```

接著，系統會開啟新視窗或分頁，載入網址 `https://developers.google.com/web`。你可以視需要變更。

```
event.waitUntil(  
  clients.openWindow('https://developers.google.com/web/')  
);
```

`event.waitUntil()` 可確保瀏覽器不會在新視窗或分頁顯示前終止 Service Worker。

馬上試試

請再次嘗試在開發人員工具中觸發推送訊息，然後按一下通知。現在你會看到通知關閉，並開啟新分頁。

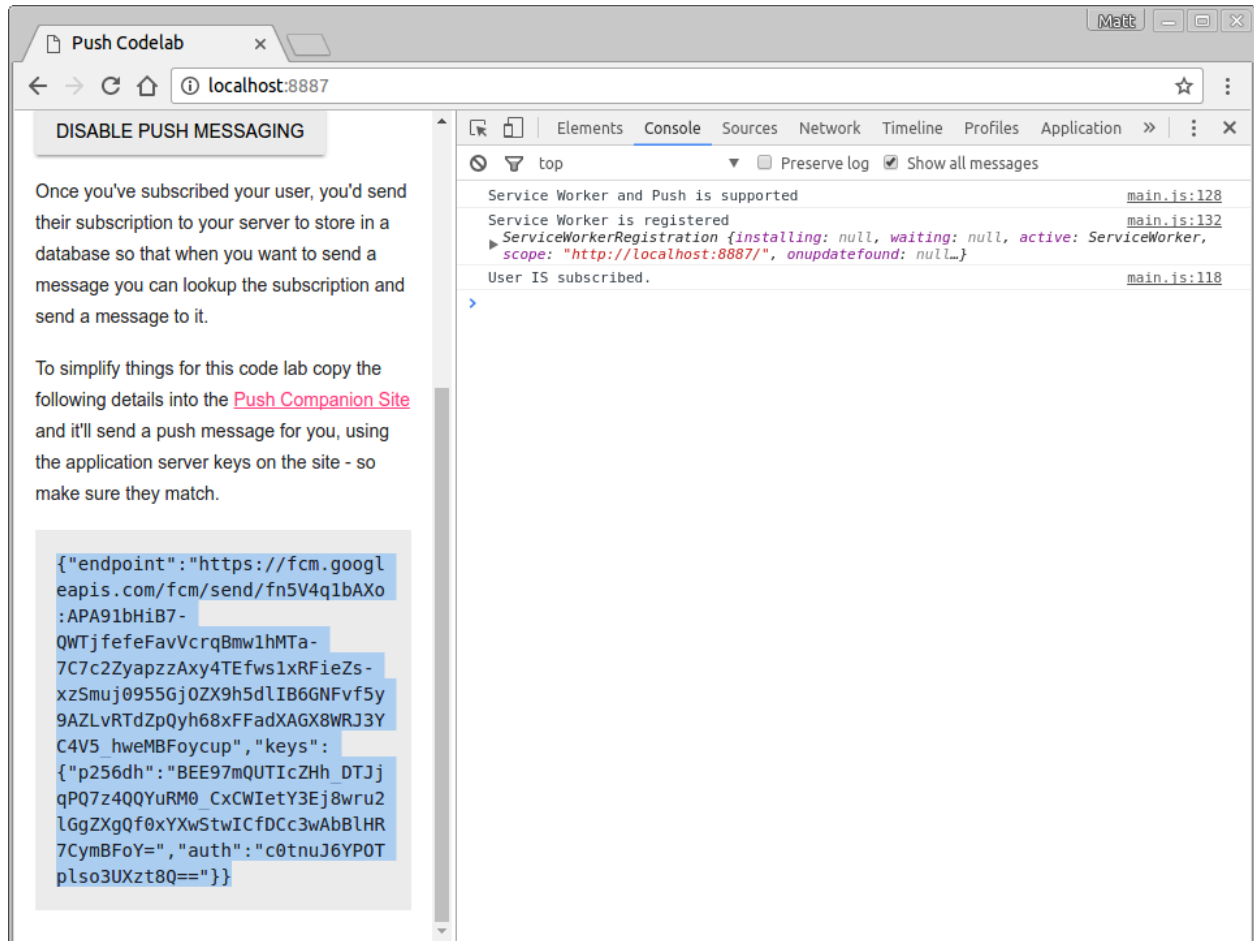
步驟 9 · 約 0 分鐘

9. 傳送推播訊息

您已瞭解如何使用開發人員工具，讓網頁應用程式顯示通知，也看過如何點按關閉通知。下一步是傳送實際的即時訊息。

一般來說，這需要從網頁將訂閱項目傳送至後端。接著，後端會對訂閱項目中的端點發出 API 呼叫，觸發推送訊息。

這不是本程式碼實驗室的學習範圍，但您可以使用隨附網站 (web-push-codelab.glitch.me) 觸發實際的推播訊息。將訂閱項目貼到頁面底部：



然後將這段文字貼到隨附網站的「Subscription to Send To」(要傳送至的訂閱項目) 文字區域：

Push CodelabPush Companion

Secure | https://web-push-codelab.appspot.com

Codelab Message Sending

When you are going through our [push codelab](#) you can send a message below.

Subscription to Send To

```
{ "endpoint": "https://fcm.googleapis.com/fcm/send/cyl6lFlpczc:APA91bE8ShJ_3ZiKVKLmuKN6oJB0yX5pU6vB0pn7WirBgNEz6iG8p-T1lGkAh_ITPPjLkW3pXFZd9XG5hW9FZ0lUixh0l49qTLJfM0g7ygEiakCvMQDxND6GkT9BhDZPni5Z62o2pmyb", "keys": { "p256dh": "BMzkuaA2W9Sehe5ImR8-ry8k2SZfqVw_h6lWWk4JWcYc74DVuH_0k9AK8ThvrRWCZ9zN9v0-IJ1s_moKPZDiOc=", "auth": "VXYiHGS3KqilOW1QzFWCaQ==" } }
```

Text to Send

Add Push Text Here....

SEND PUSH MESSAGE

在「要傳送的文字」下方，新增要隨推播訊息傳送的任何字串。

按一下「傳送推播訊息」按鈕。

Push CodelabPush Companion

Secure | https://web-push-codelab.appspot.com

Codelab Message Sending

When you are going through our [push codelab](#) you can send a message below.

Subscription to Send To

```
{ "endpoint": "https://fcm.googleapis.com/fcm/send/cyl6lFlpczc:APA91bE8ShJ_3ZiKVKLmuKN6oJB0yX5pU6vB0pn7WirBgNEz6iG8p-T1lGkAh_ITPPjLkW3pXFZd9XG5hW9FZ0IUixh0l49qTLJfM0g7ygEiakCvMQDxND6GkT9BhDZPni5Z62o2pmyb", "keys": { "p256dh": "BMzkuaA2W9Sehe5ImR8-ry8k2SZfqVw_h6IWWk4JWcYc74DVuH_0k9AK8ThyrRWCZ9zN9y0-lJ1s_moKPZDiOc=", "auth": "VXYiHGS3KgilOW1QzFWCaQ==" } }
```

Text to Send

This is some example test, but this could be a [JSON](#) string.

SEND PUSH MESSAGE

接著，您應該會收到推送訊息。您使用的文字會記錄到控制台。

Push CodelabPush Companion

localhost:8887

Once you've subscribed your user, you'd send their subscription to your server to store in a database so that when you want to send a message you can lookup the subscription and send a message to it.

To simplify things for this code lab copy the following details into the [Push Companion Site](#) and it'll send a push message for you, using the application server keys on the site - so make sure they match.

```
{ "endpoint": "https://fcm.googleapis.com/fcm/send/cyl6lFlpczc:APA91bE8ShJ_3ZiKVKLmuKN6oJB0yX5pU6vB0pn7WirBgNEz6iG8p-T1lGkAh_ITPPjLkW3pXFZd9XG5hW9FZ0IUixh0l49qTLJfM0g7ygEiakCvMQDxND6GkT9BhDZPni5Z62o2pmyb", "keys": { "p256dh": "BMzkuaA2W9Sehe5ImR8-ry8k2SZfqVw_h6IWWk4JWcYc74DVuH_0k9AK8ThyrRWCZ9zN9y0-lJ1s_moKPZDiOc=", "auth": "VXYiHGS3KgilOW1QzFWCaQ==" } }
```

ElementsConsoleSourcesNetworkTimelineProfilesApplication

top

Preserve logShow all messages

[Service Worker] Push Received. sw.is:25

[Service Worker] Push had this data: "This is some example test, but this could be a JSON string." sw.is:26

您應該有機會測試傳送及接收資料，並據此操控通知。

這個隨附應用程式只是使用 [web-push 程式庫](#) 傳送訊息的節點伺服器。建議您查看 [GitHub 上的 web-push-libs 機構](#)，瞭解有哪些程式庫可供您傳送推播訊息。這會處理許多觸發推播訊息的詳細資料。

您可以在[這裡](#)查看隨附網站的所有程式碼。

10. 取消訂閱使用者

完成的程式碼

但缺少取消訂閱使用者推播通知的功能。如要這麼做，您需要在 `PushSubscription` 上呼叫 `unsubscribe()`。

返回 `scripts/main.js` 檔案，將 `initializeUI()` 中的 `pushButton` 點擊事件監聽器變更為下列項目：

```
pushButton.addEventListener('click', function() {
  pushButton.disabled = true;
  if (isSubscribed) {
    unsubscribeUser();
  } else {
    subscribeUser();
  }
});
```

請注意，您現在要呼叫新函式 `unsubscribeUser()`。在這個函式中，您會取得目前的訂閱項目，並對其呼叫 `unsubscribe()`。在 `scripts/main.js` 中加入下列程式碼：

```
function unsubscribeUser() {
  swRegistration.pushManager.getSubscription()
    .then(function(subscription) {
      if (subscription) {
        return subscription.unsubscribe();
      }
    })
    .catch(function(error) {
      console.log('Error unsubscribing', error);
    })
    .then(function() {
      updateSubscriptionOnServer(null);

      console.log('User is unsubscribed.');
```

```
      isSubscribed = false;

      updateBtn();
    });
}
```

我們來逐步瞭解這個函式。

首先，請呼叫 `getSubscription()` 取得目前的訂閱項目：

```
swRegistration.pushManager.getSubscription()
```

如果存在 `PushSubscription`，這會傳回以該物件解析的 `Promise`；否則會傳回 `null`。如果有訂閱項目，您會對其呼叫 `unsubscribe()`，這會使 `PushSubscription` 無效。

```
swRegistration.pushManager.getSubscription()
  .then(function(subscription) {
    if (subscription) {
      // TODO: Tell application server to delete subscription
      return subscription.unsubscribe();
    }
  })
  .catch(function(error) {
    console.log('Error unsubscribing', error);
  })
```

呼叫 `unsubscribe()` 會傳回 Promise，因為這項作業可能需要一段時間才能完成。您會傳回該 Promise，因此鏈結中的下一個 `then()` 會等待 `unsubscribe()` 完成。您也可以新增 `catch` 處理常式，以防呼叫 `unsubscribe()` 時發生錯誤。完成後，您就可以更新 UI。

```
.then(function() {
  updateSubscriptionOnServer(null);

  console.log('User is unsubscribed.');
```

```
  isSubscribed = false;

  updateBtn();
})
```

馬上試試

您應該可以在網頁應用程式中按下「啟用推播訊息」或「停用推播訊息」，記錄檔會顯示使用者訂閱及取消訂閱的狀態。

Push Codelab

Push Companion

localhost:8887

Service Worker and Push is supported

Service Worker is registered

ServiceWorkerRegistration {installing: null, waiting: null, active: ServiceWorker, scope: "http://localhost:8887/", onupdatefound: null...}

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is unsubscribed.

User is subscribed.

User is subscribed.

Push Codelab

Welcome to the push messaging codelab. The button below needs to be fixed to support subscribing to push.

DISABLE PUSH MESSAGING

Once you've subscribed your user, you'd send their subscription to your server to store in a database so that when you want to send a message you can lookup the subscription and send a message to it.

To simplify things for this code lab copy the following details into the [Push Companion Site](#) and it'll send a push message for you, using the application server keys on the site - so make sure they match.

11. 已完成

恭喜您完成本程式碼研究室！

本程式碼研究室已說明如何開始在網頁應用程式中新增推播通知。如要進一步瞭解網頁通知的功能，請參閱[這些文件](#)。

如要在網站上部署推播通知，您可能會想新增對舊版瀏覽器或使用 GCM 的非標準相容瀏覽器的支援。請按[這裡](#)瞭解詳情。

其他資訊

- [網路推播通知](#)：網站基礎知識說明文件。
- [網頁推送程式庫](#)：網頁推送程式庫，包括 Node.js、PHP、Java、Python、C 和 C#。

相關網誌文章

- [網頁推送訊息酬載加密](#)
 - [應用程式伺服器金鑰和網頁推送通知](#)
 - [通知動作](#)
 - [圖示、關閉活動、重新通知偏好設定和時間戳記](#)
-